

状态机:

(nPC,R0,R1,R2)

Rom:

0:lim r0 'd1

1:lim r2 'd100

2:lim r1 'd1

3:addi r0+1=>r0

4:add r1+r0=>r1

5:jalr r0!=100?=>3:=>6

6:ebreak

0, x,x,x

1,1,x,0

2,1,x,100

3,1,1,100

4,2,1,100

5,2,3,100

3,2,3,100

4,3,3,100

5,3,6,100

.....

3,99,4950,100

4,100,4950,100

5,5500,100

6,5500,100

Ebreak

关于基础设施:

如果我仅仅是使用 GDB, 并且假设每一次调试解决一个 BUG, 那需要 $450 \times 20 \times 30 \Rightarrow 75$ 个小时, 而在使用 sdb 之后就变成了 $1/3$ 也就是 25 个小时, 节省的时间显著。

riscv32 的 RTFM

1.riscv32 有哪几种指令格式

=>2.2. Base Instruction

Formats,p23

2.LUI 指令的行为是什么?

=>2.4.1. Integer Register-

Immediate Instructions,p26

3.mstatus 寄存器的结构是怎么样的?

=>特权手册的 3.1.6. Machine Status Registers (mstatus and mstatush)

shell 命令:

在 PA0 的时候的代码量是 259909 行, 排除空行是 224650 行。而 PA1 的代码量是 267424 行, 排除空行后是 231100 行。

RTFM(-Wall 和 -Werror)

-Wall 的作用是显示所有的警告, 而 -Werror 是把所有的警告视为错误。使用 -Wall 和 -Werror 的作用是强制显示所有的可能的错误并全部视为不可通过的错误, 就可以强制要求我们去解决所有编译器能够检测出来的潜在问题否则就不允许通过编译, 这样子就可以强制我们提高我们的代码质量, 在一定程度上减少 BUG。